Apple Keynote .key 文件格式分析报告

- **分析对象**：（用户自建 主题，465,397 字节）
- **分析环境**：macOS / Keynote 14.5（Build 7045.0.17）
- **分析方法**： + 自建字节级解码器 + 官方 protobuf schema（keynote-parser 1.14.5 的 v14_5 定义）交叉验证，全部结论以 Keynote 实机打开为验收判据
- **报告日期**：2026-10-16（2026-10-17 增补「颜色改写」）

一、结论摘要

.key 是一个**可完整解包、可语义解析、可按页寻址、可安全回写**的复合格式。分析已推进到七层：

层级	格式	状态
容器层	Zip 归档（全部条目 ZIP_STORED）	已实测确认
数据层	IWA：0x00 + 小端 uint24 + Snappy 块（可多 chunk）	已实测确认
段层	IWAArchiveSegment 序列（protobuf）	已实测确认
语义层	对象类型与内容（631 种类型的映射）	已实测确认
引用图层	**Show→SlideNode→Slide→文本/备注 的完整寻址链**	**已实测确认**
状态开关层	**页面级开关（isSkipped 跳过放映等）**	**已实测确认**
对象内容层	**表格文字 / 数值、构建动画、主题与元素颜色的读写**	**已打通**

已具备且经 Keynote 实机验证的能力：

- 解包、读取全部结构与媒体资源
- **按语义识别每一个对象**：1360 个对象全部解析成功，0 失败、0 未知类型
- **提取文档里的真实文字**（标题、正文、页脚、版式名等）
- **按页导出大纲**：标题 / 正文 / 演讲者备注（实测 21 页 deck 全部读出）
- **等长替换**文本
- **变长替换**文本（长度链全自动重算）
- **跨全部 .iwa 批量替换**（实测一次改 17 个文件、17 处页脚）
- **追加演讲者备注**（按页追加段落，Keynote 实机确认可读；既有备注不被改写）
- **跳过 / 取消跳过指定页**（支持 1-3,7 / all 批量，Keynote 实机确认）
- **整段改写标题 / 正文**（沿引用链改 StorageArchive.text，非子串替换）
- **改写既有备注**（整段替换 / --replace 子串替换）
- **备注合规体检**（批量查缺备注、缺【内容依据】【设计依据】）
- **替换媒体文件**（同名字节替换，无需同步元数据）
- **表格文字写入**（Index/Tables/ 的字符串表，明文可改）
- **构建动画读写**（列出各页动画；改时长与自动播放）

- **综合体检**（audit：备注 / 跳过 / 动画 / 表格，一次看全）
- **表格数值写入**（cell_storage_buffer 的缩放整数，Keynote 读回 0.777）
- **颜色读写**（枚举 / 按序号改 / 改页背景 / 改形状填充；Keynote + PNG 像素双验证）
- **页级精确改文字色**（新建 CharacterStyle 挂 table_char_style，不污染共享样式）
- **增删幻灯片**（slideTree.slides；删除时级联清理对象）
- **增删形状**（owned_drawables + drawables_z_order；删除时级联清理 storage/caption）
- **改写既有备注**（set-note，覆盖式）

结构编辑：增删幻灯片 / 形状、页级精确改文字色均已打通；跨大版本仍需实机复核。

二、第一层：Zip 容器

.key 本质是 Zip，file 判定为 Zip archive data, compression method=store。 实测 含 **60 个条目**，无符号链接：

> Index/*.iwa 31 个 —— 文档与幻灯片的真正数据
> Data/*.jpg 24 个 —— 媒体素材
> Metadata/Properties.plist / DocumentIdentifier / BuildVersionHistory.plist
> preview.jpg / preview-web.jpg / preview-micro.jpg

全部条目 ZIP_STORED（不压缩），重打包无需处理压缩差异。

三、第二层：IWA 数据格式

> 一个 .iwa = 1 个或多个串联 chunk
> 每个 chunk = [0x00] [压缩长度 little-endian uint24] [Snappy 数据]

文件	大小	chunk 数	解压后
Document.iwa	7,751	1	14,705
Slide.iwa	1,480	1	3,004
DocumentStylesheet-3843.iwa	47,811	**5**	—

要点：长度字段是**小端 uint24**；**一个 .iwa 可含多个 chunk**；Snappy 是 **raw/block 格式**（无 sNaPpY 魔数）。

四、第三层：段结构

这是最容易搞错的一层。早先按"每条消息都有 varint 长度前缀"切分会立刻失败，真正规则是：

> Snappy 解压后（多 chunk 拼接） = 若干 IWAArchiveSegment 顺序排列
>
> IWAArchiveSegment =

```
[varint: ArchiveInfo 的字节数]    ← ★只有这里有长度前缀
[ArchiveInfo]
[object_0][object_1]...           ← ★裸拼接，靠 length 定界

ArchiveInfo = { field 1: identifier, field 2: message_infos[] }
MessageInfo = { field 1: type, field 2: version,
                field 3: length ★该 object 的字节数，唯一定界依据★,
                field 4/5: 引用列表 }
```

只有 ArchiveInfo 带长度前缀；object 靠 MessageInfo.length 定界。

实测印证：Slide.iwa 第 1 段 ArchiveInfo 长 60 字节，其 MessageInfo.length = 180，紧随的裸数据正好 180 字节。

全量验证：31 个 .iwa 全部干净解析到末尾，累计 **1354 段 / 1360 个对象**，零失败。

文件名规律：.iwa 文件名里的数字就是文件内 ArchiveInfo.identifier（Document.iwa=1、Metadata.iwa=2、TemplateSlide-1610803-2.iwa=1610803）。

五、第四层：对象语义（本次打通）

有了段的粒度还不够——必须知道"每个 object 到底是什么"。Keynote 用 MessageInfo.type 编号标识对象类型，这个映射表从未公开，本次通过 keynote-parser 1.14.5 的 **v14_5** 定义取得并固化。

映射表规模：ID_NAME_MAP **631** 条、NAME_CLASS_MAP **1359** 条。 版本对齐：本机 Keynote **14.5** ⟷ schema **v14_5**。

关键类型（与实测吻合）：

type	类	说明
1	KN.DocumentArchive	对应 Document.iwa 的 identifier=1
2	KN.ShowArchive	对应 Metadata.iwa 的 identifier=2
4	KN.SlideNodeArchive	幻灯片树节点
5 / 6	KN.SlideArchive	幻灯片本体
9	KN.SlideStyleArchive	幻灯片样式
14	TSWP.TextualAttachmentArchive	文本附件
15	KN.NoteArchive	**演讲者备注**

全量语义解析结果：　　的 **1360 个对象全部解析成功**（0 失败 / 0 未知类型），类型分布：

```
360  TSD.StandinCaptionArchive
205  TSWP.StorageArchive        ← 文字容器
149  TSWP.ParagraphStyleArchive
102  TST.CellStyleArchive
101  TSWP.ShapeInfoArchive
```

```
  95  KN.PlaceholderArchive
  41  TSWP.ShapeStyleArchive
  36  TSCH.ChartSeriesStyleArchive
  24  KN.SlideNodeArchive
  24  KN.SlideArchive          ← 24 张幻灯片
  24  TSD.GuideStorageArchive
  18  TSCH.ChartAxisStyleArchive
  18  KN.MotionBackgroundStyleArchive
```

六、引用图：把「第 N 张幻灯片」串起来

有了对象类型还不够——真正的文档结构藏在 TSP.Reference 构成的引用图里。本次沿引用链走通了完整路径：

```
KN.ShowArchive (field 3: slideTree)          ← 位于 Index/Document.iwa
  └ KN.SlideTreeArchive.slides[]             (内联消息，repeated TSP.Reference)
    └ KN.SlideNodeArchive.slide              (TSP.Reference)
      └ KN.SlideArchive
        ├ .titlePlaceholder / .bodyPlaceholder   (TSP.Reference)
        │   └ KN.PlaceholderArchive.super        ★内联 TSWP.ShapeInfoArchive
        │     └ ShapeInfoArchive.owned_storage   (TSP.Reference)
        │       └ TSWP.StorageArchive.text       (repeated string)
        ├ .owned_drawables[] → 同上（TSD.GroupArchive.children 需递归）
        └ .note                                  (TSP.Reference)
          └ KN.NoteArchive.containedStorage      (TSP.Reference)
            └ TSWP.StorageArchive.text           ← 演讲者备注
```

三个曾卡住的点（都属于"看着像引用、实际是内联"的坑）：

1. ShowArchive.slideTree 是**内联消息**，不能再往下一层取 .identifier；
2. 占位符指向的是 **KN.PlaceholderArchive**，其 super(1) 才是 TSWP.ShapeInfoArchive；
3. 文字最终挂在 **ShapeInfoArchive.owned_storage(4)**，而非 deprecated_storage(2) 或 text_flow(3)。

实测效果（真实内容 deck，21 页）：

```
--- 第 1 页 ---
  标题:
  正文: 一次真实自建的选择与权衡
  备注: 有（185 字）
--- 第 3 页 ---
  标题:
```

> 正文: ① 系统形态： / 纯 Wiki / 双轨
> 备注: 有（145 字）

附带发现：该 deck **21 页全部带演讲者备注**，且内容严格遵循「【内容依据】/【设计依据】」格式。这说明备注层不仅能读，还可被程序化校验——例如批量检查哪些页缺备注、格式是否合规。

七、文字内容：明文且可语义定位

文字存放在 TSWP.StorageArchive，其 text 是 **repeated string**（按段落存储）。实测 74 个文本对象含非空内容，例如：

对象 ID	文件	内容
1611013	TemplateSlide.iwa	标题文本
1610829	TemplateSlide.iwa	正文级别 1 / 正文级别 2 / … / 正文级别 5（5 段）
6879323	TemplateSlide.iwa	（页脚）
6682201 段	TemplateSlide–6682201.iwa	版式名 标题 – 顶部对齐_纯黑背景

全部为明文 UTF–8，无加密、无混淆，可按对象 ID 精确定位。

7.1 媒体资源（Data/）

Data/ 下的文件（实测样本：mt–* 21 个 + st–* 22 个，均为 jpg）通过 **TSP.DataReference（只有 identifier 一个字段）**被引用，主要引用方是 KN.SlideNodeArchive.thumbnails（幻灯片缩略图）。内容图片的挂载点是 TSD.ImageArchive：data(11) / originalData(13) / thumbnailData(12) → TSP.DataReference → Data/ 文件。

关键结论：媒体文件没有任何摘要或大小校验。 TSP.DataMetadata 只有一个 fallback_color 字段，不含 digest/size。因此：

替换媒体 = 直接替换 Zip 中同名条目的字节（保持文件名不变，引用即不断）。

实测替换一个 Data/mt–*.jpg 的字节后，Keynote 正常打开（slides=1, layouts=23）。

八、段落属性约束与结构化写入

前面的替换只改**字符**，段落数不变。但"给某页补写备注"会**增加段落**——这会不会破坏结构？本轮把这条约束查清了。

8.1 文本的真实组织方式

一个常见误解是把 StorageArchive.text 当作"每段一个元素"。实测并非如此：

```
repeated text 元素数 = 1
  elem[0] = '正文级别 1\n正文级别 2\n正文级别 3\n正文级别 4\n正文级别 5'
```

段落由字符串内部的 \n 分隔，而不是 repeated 元素。

8.2 段落属性表为空（关键安全结论）

StorageArchive 理论上带一批按段落索引的属性表：table_para_data(6)、table_para_style(5)、table_para_starts(14) 等。若这些表按字符偏移索引，增删段落就会使偏移错位。

全量实测：

文件	StorageArchive 数	最大段落数	table_para_* 非空的对象
模板	205	5	**0**
构选项….key	282	8	**0**

487 个对象全部为空——段落属性不在此层承载。因此在 StorageArchive 层增删段落不会破坏内部索引。

8.3 实测：追加一段备注

在真实 deck 第 1 页备注末尾追加一段：

- 备注 185 → 218 字，object 497 → 561 字节，条目负载同步重算
- **Keynote 打开正常（slides=21），presenter notes 读出 218 字，含追加=true**， 尾部即追加的文字

结论：结构化写入（增加段落）可行且不破坏文档。

这直接支撑了一条实际规范：**新生成的幻灯片应当写入备注（内容依据 / 设计依据 / 口播参考）**，且对既有备注只能**追加、不得改写**。工具据此提供 note 子命令，只做追加。

8.4 整段写入（非子串替换）

既然段落数可变是安全的，就可以做**整段重设**而不只是子串替换：沿引用链定位该页 标题/正文的 TSWP.StorageArchive，直接改写其 text 字段。实测：

- set-text 第 1 页标题 8 → 15 字，Keynote 读到的文本对象中确为改写后的标题
- set-note 第 1 页备注 185 → 36 字（**覆盖**），Keynote presenter notes 读出新内容

> 注：set-note 负责改写（整段或子串），note 负责末尾追加； 常规补充仍应使用 note（追加）。

九、页面级开关：跳过与折叠

「跳过幻灯片」（放映时不出现、但保留在文稿里）的开关位置：

KN.SlideNodeArchive.isSkipped（field 4，bool）

9.1 三向验证

方向	证据
读（Keynote → 文件）	某 deck 中 isSkipped=True 的节点（ident=7036107），正是 Keynote 的 skipped of slide 12

方向	证据
写（文件 → Keynote）	工具写入第 3 页 isSkipped=True → Keynote 报告「**被跳过的页: 3,12**」
取消（写 0）	工具对第 12 页写 isSkipped=0 → Keynote 报告「**被跳过的页: (无)**」

9.2 一个真实踩坑：布尔字段必须「显式写 0」

取消跳过的第一版实现**删除了 field 4**，结果 Keynote 打开时报 –1700 不能将 missing value 转换为 specifier（文档未能正常装载）。

字节层诊断给出了原因：

> 原始 deck 的 SlideNodeArchive 字段 4 存在性：
> 显式 =1 : 1 个
> 显式 =0 : 43 个　　← 未跳过的页也把 0 写出来了
> 省略字段: 0 个

Keynote 从不省略该字段：未跳过时也显式写出 field 4 = 0（proto2 optional 的显式存在语义）。 改为写 0 后，44 个节点全部 =0、与原格式一致，Keynote 随即正常打开并报告「(无)」。

教训：改写二进制结构时，"语义等价的省略"未必是"字节一致的合法"——必须照抄原生写法。

9.3 相邻开关：折叠状态（语义已确认）

同一节点上还有两个开关：

字段	含义	默认写法
isSkipped(4)	跳过放映	**显式写 0**（43/43 个未跳过节点都带 =0）
isCollapsed(5)	大纲视图折叠	**省略**（显式存在 = 0 个）
isCollapsedInOutlineView(17)	大纲视图折叠	**省略**（显式存在 = 0 个）

两者是不同性质的字段：isSkipped 总是被写出，而两个折叠字段是真正的 optional、 默认值下完全不出现。实测把后两者显式写入 =1，Keynote 仍正常打开（slides=21）， 说明它们可安全写入，但**语义属大纲视图的 UI 状态，不影响放映**——与 isSkipped 不同。

这一差异正是 9.2 那条教训的根源：改动某个 bool 字段前，必须先看清它是"总是写出" 还是"默认省略"，不能按语义想当然。

十、对象级内容：表格、图表、动画与主题

10.1 表格（TST.*）——文字可写

表格数值的真实落点（实测于 TableSample.key）：

> TST.TableModelArchive
> ├── number_of_rows / number_of_columns

```
└── base_data_store.tiles.tiles[].tile → TST.Tile
    └── rowInfos[行].cell_storage_buffer
        ├── cell_offsets: uint16 小端数组，第 i 项 = 第 i 个单元格起点
        ├── 数字格：36 字节，**值在 +12 的小端 int32**
        └── 文本格：24 字节（指向 stringTable）
```

实测：TableSample.key 是 5 行 × 4 列，行 2 = [2, 4, 4, 6]。 把 B2 的 04 00 00 00（+12 偏移）改成 99 03 00 00 后 Keynote 读回 **999.0**：

```
$ set-cell TableSample.key out.key 7031011 2 2 777
表格 7031011 第 2 行第 2 列：4 → 777
→ Keynote 读回：B2=777.0；C2=4.0（未动）；A2=2.0（未动）
```

增删行列仍未打通：行数不是由 TableModel.number_of_rows 单独决定，TST.ColumnRowUIDMapArchive 里维护着 sorted_row_uids（按 UUID 排序）、 row_index_for_uid、row_uid_for_index 三份互相一致的索引。 只追加而不保持排序会让 Keynote 拒绝打开。

10.2 图表（TSCH.*）——数值可写（已验证）

图表由 **TSCH.ChartDrawableArchive** 表示（只有 super → TSD.DrawableArchive： 几何、换行、标题/说明引用），其非样式部分 TSCH.ChartNonStyleArchive 存放在 **Index/ObjectContainer.iwa**（该条目的存在即"此 deck 有图表"的标志； 实测 265 个 deck 中 80 个含它）。

TSCH.ChartDrawableArchive 与 ChartNonStyleArchive **都不承载数值**。 Keynote 图表的数据源与表格同构——数值以 **TST.TableDataList + cell_storage_buffer 的缩放整数**形式存在（见 10.1），因此**图表数值可用与表格相同的方法改写**； TSCE 公式引擎只在单元格使用公式时才介入（实测 deck 有 22 个 TSCE.FormulaOwnerDependenciesArchive）。

图表数据的真实落点：不是独立的 TSCH.ChartArchive 对象，而是内联在 TSCH.ChartDrawableArchive.unity（类型 TSCH.ChartArchive）的 grid 里：

```
chart_type: columnChartType2D
grid {
  row_name:    ["区域 1", "区域 2"]
  column_name: ["四月", "五月", "六月", "七月"]
  grid_row[{ value[{ numeric_value: 17 }, …] }, …]
}
```

实测（T.key，2 行 × 4 列柱状图）：

```
$ charts T.key
--- 图表 7031018（2 行 × 4 列）
  行: ['区域 1', '区域 2']  列: ['四月', '五月', '六月', '七月']
  行0: [17.0, 26.0, 53.0, 96.0]
  行1: [55.0, 43.0, 70.0, 58.0]
```

```
$                          set-chart T.key out.key 7031018 0 0 100
图表 7031018：区域 1 / 四月 17.0 → 100.0
```

Keynote 打开 + 导出 PNG：非白采样 12749 → 15059，两图差异 9540 采样点 （柱高随 100 > 17 变化）。写回后读回数值也确认一致。

10.3 构建动画（KN.Build*）——可读可写

一页的动画由 KN.SlideArchive.builds(2) 与 buildChunks(43) 引用：

- **KN.BuildArchive**：drawable(1) 作用对象、delivery(2) 投递方式、 attributes(4) → BuildAttributesArchive（animationAttributes 效果参数、 eventTrigger 触发方式、delayAutomatic* 等）
- **KN.BuildChunkArchive**：build(1) → 所属构建、delay(3) **double**、 duration(4) **double**、automatic(5) bool

实测（ .key，8 页含动画）读出：

```
第1页  构建 投递=All at Once  effect=apple:bc-appear
       块 delay=0.0 时长=0.5 自动=False
第9页  构建 effect=apple:movie-start / apple:bc-appear
       块 时长=1.0 自动=False ； 时长=0.5 自动=True
```

实测写入：把第 1 页某块 时长 0.5 → 3.0、自动 False → True，回读一致， **Keynote 打开正常（slides=31）**。

> 踩坑：delay/duration 是 **double（wire type 1）**，不是 float（wire type 5）。 按 float 写入会静默失败（回读值不变，但不报错）——**写标量也要先核对类型**。

10.4 母版与主题颜色（已打通：三大落点 + 实测）

```
KN.ThemeArchive.super
  ├ color_presets[30]                          ← 主题调色板
  └ extension.gradient_fill_presets[] / image_fill_presets[]
KN.SlideStyleArchive
  ├ super                                      → TSS.StyleArchive
  └ slide_properties(11) → fill(1) → color(1)      ← 页背景
TSWP.ShapeStyleArchive
  └ super.shape_properties → fill → color          ← 形状填充
TSWP.ParagraphStyleArchive
  └ char_properties → font_color / tsd_fill.color  ← 文字颜色（两处同值）
```

TSP.Color 不是独立对象，而是内联子消息。上一版报告写「计数为 0」，是因为只查了 顶层对象列表；对整棵消息树递归下钻后， 模板有 **2939** 个实例、母版模板 2000 余个：

落点	数量级	语义
TSCH.* 图表样式	~1700	图表系列默认色（样式库预设）
TST.* 表格样式	~650	表格描边 / 单元格填充预设
TSWP.ParagraphStyleArchive	~140	段落文字色
KN.ThemeArchive.color_presets	30	主题调色板（白/灰/黑/蓝/青/绿/黄/橙/红/粉）
KN.SlideStyleArchive	每页 1	页背景
TSWP.ShapeStyleArchive	每形状 1	形状填充 / 描边 / 阴影

颜色值是 sRGB 0..1：model=rgb(1)、rgbspace=srgb(1)，r/g/b/a 为 float（wire 5）。 注意 AppleScript 的 color 是 0..65535 的**显示空间**：在 P3 屏上设纯红 {65535,0,0}， Keynote 存进文件的是 (1.0, 0.149, 0.0)——P3→sRGB 换算的结果。**写入直接用 sRGB 原值。**

三种颜色路径的实机验证（Keynote 打开 + 导出 PNG 取像素，双证据）：

改法	改的东西	证据
文字色	单页样本 font_color 绿→紫	PNG 中绿色消失，出现 (6,51,251)
页背景	第 2 页 SlideStyleArchive.fill.color 白→红	PNG 第 2 页 (252,59,47)，第 1 页仍白
形状填充	母版模板样式 734 红→绿	PNG 中 9 页红块全部变绿，像素数与原来吻合

一个必须知道的语义陷阱：TSS.StylesheetArchive（母版模板里 569 个样式）是**样式库**， 其中大量样式**没有被任何页面引用**。改「未引用的副本」→ 界面毫无变化（实测踩过两次）； 改「被引用的样式」→ 所有引用它的页一起变色（**这才是「改母版 / 主题色」的正确语义**）。 因此工具会先 colors 列出实例，并在 set-shape / set-bg 时提示该样式被多少页共用。

颜色相关命令：

```
# 列出全部颜色实例（默认隐藏图表/表格的样式库预设，--all 全显示）
                        colors deck.key

# 按序号改一个颜色（序号来自 colors 的输出）
                        set-color src.key dst.key 1943 '#FF3B30'

# 改第 2 页纯色背景 / 第 3 页第 1 个形状的填充
                        set-bg    src.key dst.key 2 '#FF3B30'
                        set-shape src.key dst.key 3 0 '#FF3B30'
```

> 改「被引用的样式」会同时影响所有引用它的页（工具会打印影响页数）； 改「未被引用的样式副本」界面无变化——这两种情况都实测过。

10.5 综合体检流水线

audit 把上述维度合到一条命令里（只读）：

```
$                              audit                 ….key
总页数 21
  备注合规 21 页 | 缺备注 0 | 字段不全 0 | 被跳过 0 | 含动画 0 | 表格 1

$                              audit                 .key
总页数 31
  备注合规 0 页 | 缺备注 23 页 [2,3,4,…] | 字段不全 8 页
  被跳过 0 | 含构建动画 8 页 第1页(1构建/1块); 第2页(1构建/3块); …
  表格 0
```

同一份实现既能确认"规范 deck 达标"，也能在历史 deck 上暴露 **23 页缺备注、8 页字段不全、8 页带动画**。

十一、结构编辑：增删幻灯片与形状（已打通）

11.1 页序的唯一权威是 ShowArchive.slideTree.slides

```
ShowArchive.slideTree(内联 SlideTreeArchive)
  └ slides[] = repeated TSP.Reference → SlideNodeArchive
      └ slide(2) → SlideArchive
```

rootSlideNode 实测**未被设置**，页序完全由 slides 列表决定。因此：

- **删除幻灯片** = 从 slides 移除 reference（再级联删对象）。
- **插入幻灯片** = 在 slides 里插入新的 reference。

11.2 按版式新增页（深拷贝）

版式在文件里就是一组 SlideArchive（template=False，名字如「标题项目符号」）， 用户页通过 SlideArchive.template_slide 指向其一。

add-slide <版式名> [位置] 的做法：

1. 按 SlideArchive.name 找到版式页；
2. 深拷贝它的整个闭包——PlaceholderArchive / ShapeInfoArchive / StorageArchive / GroupArchive / StandinCaptionArchive 全部换新 ident，**样式与主题保持共享**；
3. 新建 SlideNodeArchive，插入 slideTree.slides。

实测： 模板 add-slide "标题项目符号" → 2 页、深拷贝 24 个子对象； 改新页标题后旧页仍为「手打啥」、新页为「新页标题」。

> 注意 **硬限制（已定论）**：新增/复制的页在 Keynote 中**无效**—— 页数统计得到 2，但设置 current slide 会报「无法将当前幻灯片设为不在该演示文稿中的幻灯片」， 导出 PNG 为空白。已系统排查并逐一排除： template_slide、placeholder 结构、parent 指向、样式引用、缩略图（含真实缩略图）、

SlideNode UUID、模板专属字段、共享背景对象、对象写入的 .iwa 归属。 结论：Keynote 对「新页」有额外的文档模型登记要求，纯文件层无法可靠伪造。

本轮补充的结构知识（对后续逆向有价值）：

1. **每页对象放在独立 iwa**：Keynote 新建页会写一个 Index/Slide-<页ident>.iwa， 内含 15 个对象，每对象一段：SlideArchive ×1、PlaceholderArchive ×3、 StandinCaptionArchive ×4、StorageArchive ×4、NumberAttachmentArchive ×1、 GuideStorageArchive ×1、NoteArchive ×1。而 Slide.iwa 只装原有页。
2. **MessageInfo 有五个字段**：type(1)、**version(2)**、length(3)、 **references(4/5)**。只写 type+length 的段结构能被解析，但 Keynote 不接受； 补上 version(01 00 05) 与 references 后仍无效，说明 references 的语义 （全局 ident 还是本地索引）尚未完全解出。
3. **ViewState 会重命名**：另存后 Index/ViewState.iwa 变成 Index/ViewState-<ident>.iwa。
4. **决定性反证**：把 Keynote 自己生成的整页 iwa **原样**复制进另一个文件 （连 ident 都不换），页数可见但 current slide 失败、导出空白—— 所以缺的不是页对象，而是文档级的登记（未在公开结构中找到）。 **删除页不受影响**：删页后其余 21 页渲染正常（非白采样点 1944/1669/2594）。 生产建议：**删除用本工具，新增/换版式走流程脚本 keynote_add_page.py**； add-slide/dup-slide/change-layout 保留为研究用实验命令，不要用于交付。

流程脚本（推荐）：新增页交给 Keynote 本体完成，其余仍用本工具：

```
keynote_add_page.py deck.key out.key "标题项目符号" \
  --title "新章节" --body "正文" --after 3
```

实测：--title 生效、页 2 渲染正常（非白采样 926）、--after 1 插到第 1 页之后正确。 版式名不存在时会列出全部可用版式。

换版式（--replace N）：在 Keynote 里「新增目标版式页 → 迁移标题/正文 → 删旧页」， 一步完成，页序与内容都保留。实测：

```
单页：1 页，layout=标题项目符号，标题「手打啥」保留，渲染 468
三页：版式依次为 金句 / 标题项目符号 / 空白无页码，渲染 1459 / 458 / 122
```

「换版式」的建议做法是**删旧页 + 按新版式新增页**：直接改已有页的 template_slide 会让 Keynote 拒绝打开（placeholder 结构与版式页不一致），实测风险高。

11.3 分组 / 拆组（渲染已验证）

TSD.GroupArchive 只有两个字段：super（ContainerArchive：geometry、parent、children...） 和 children（repeated Reference）。

打组三要点（缺一个渲染就掉东西）：

1. 新建 GroupArchive，geometry 取子形状的**并集**；
2. **子形状的 parent 改成组**（否则 Keynote 仍按页层级找，找不到）；
3. **z-order 里组必须留在第一个子形状原来的位置**（放到末尾会把背景类形状抬到最上层，遮挡一切）。

实测（母版模板第 1 页，7 个形状取前 2 个打组）：

> group → 页 drawables 7 → 6，渲染非白采样 1944，与原图差异 0
> ungroup → 页 drawables 恢复到 7，渲染 1944，与原图差异 0

拆组时同样要把子形状的 parent 改回页、kids 插回组原来的 z-order 位置—— 只做「children 移回页」会让渲染从 1944 掉到 629。

11.4 删除形状必须级联删除「独占子对象」

实测 Keynote 删除一个形状时，会连同它的配套对象一起删除：

```
TSWP.ShapeInfoArchive
  ├ owned_storage / deprecated_storage → TSWP.StorageArchive（文字容器）
  └ super.super.title / caption        → TSD.StandinCaptionArchive
```

只把引用从 owned_drawables / drawables_z_order 移除、保留对象段，Keynote 会**拒绝打开**； 把 owned_storage、title、caption 一并删除后正常。style（共享）和 parent（指向页）**不能删**。

11.5 页级精确改文字色：字符级覆盖（新建 CharacterStyle）

文字颜色默认挂在共享的 ParagraphStyleArchive 上（母版模板里一条段落样式被 134 个文本共用）。 要做到"只改这一页、且不动段落排版"，正确做法是**字符级覆盖**：

1. 新建 TSWP.CharacterStyleArchive（新 identifier）， char_properties 复制该页段落样式的完整属性（字体/字号），只覆盖 font_color 与 tsd_fill.color；
2. super.name 必须写（如「自定义颜色 #FF00FF」），并登记进 shared TSS.StylesheetArchive.styles；
3. 把该 Storage 的 table_char_style.entries 设为 [{character_index: 0, 样式}]——**不要写闭合项**；
4. 段落样式引用保持原样，共享样式零改动。

三个实测踩过的坑：

坑	现象	正解
super.parent 指向段落样式	样式被 Keynote 静默忽略，颜色不生效	parent 必须是另一个 CharacterStyleArchive（类型不同即失效）
没登记进 stylesheet.styles	同样不渲染	新建样式必须追加到 shared stylesheet 的 styles[]
写 character_index == len(text) 的闭合项	整条 char_style 被丢弃	只写 {0, 样式}，让区间延续到末尾
只复制本层 char_properties	文本框不渲染（段落样式常常只带 font_size，字体/颜色在父样式）	沿 super.parent 链合并整条继承链的属性再覆盖颜色

文本框（text item）同样支持：which 除了 title/body，还可以写 drawable 路径 （2、群组内 3/1）。真实文件里 53 处带色字符样式**全部挂在 ShapeInfo（文本框）上**， 说明字符级颜色本来就是给文本框用的：

```
$ set-text-color out.key 1 2 '#FF00FF'
第 1 页 2 文字颜色 → #FF00FF
  新建字符样式 ident=7027964（字符级覆盖，不修改共享样式）
→ 导出 PNG：3211 像素变化，其中品红 1730
```

局部改色：--substr 文字 只改第一次出现的子串，--range a:b 只改 [a,b)（1 基）。 区间表的写法是「在 start 处插入新样式、在 end 处恢复原样式（end 等于文本长度时省略）」， 并且 **entries 必须以 character_index = 0 开头**——只写 {1, 样式} 不生效，补上 {0, 默认} 后才渲染：

写法	PNG 结果
--substr 声（3 字中改 1 字）	1104 像素变化、品红 587
--range 1:2（只改首字）	751 像素变化、品红 406
整段（默认）	3211 像素变化、品红 1730

实测（两页样本）：

```
第 1 页（空白）：品红 0
第 2 页（改色）：品红 1655
第 3 页（对照）：品红 0
共享段落样式 1611008 的 font_color 仍为 (0,0,0)
新字符样式：name="自定义颜色 #FF00FF"，parent=1611307，color=(1,0,1)
```

Keynote 自己给**整段**改色时并不用字符样式，而是复制段落样式变体（para entries: [(0, 7028480)]、 char entries: None）；字符级样式主要用于**局部文字**强调（ 中有 53 处真实引用）。 两条路径工具都支持：默认 --char，--para 走段落样式副本。

11.6 实测结果

操作	结果
del-slide 5（母版模板）	22 → 21 页，Keynote 正常打开
dup-slide 1	22 页（新增 1 页），可打开；与原页共享绘制对象
add-slide "标题项目符号"	1 → 2 页；深拷贝 24 个对象；改新页标题不影响旧页
del-shape 1 3	第 1 页形状 3 → 2，Keynote 正常
dup-shape 1 0	第 1 页形状 3 → 4，Keynote 正常
set-text-color 2 title #FF00FF（两页样本）	第 2 页 1655 品红像素，第 1/3 页 0；共享段落样式仍为黑色
set-text-color 1 2 #FF00FF（文本框路径）	3211 像素变化、品红 1730；其余元素不动

操作	结果
set-text-color 1 2 #FF00FF --substr 声	只改目标字：1104 像素变化、品红 587
set-text-color 1 2 #FF00FF --range 1:2	只改首字：751 像素变化、品红 406
set-note ... --replace 第二段	第一段。第二段。 → 第一段。已替换。（其余保留）
set-note（空备注 / 已写备注）	0→7 字、7→8 字，Keynote 读回第二次内容

11.7 损坏文件的诊断边界

三类损坏的实测表现：

损坏方式	verify 的表现
Zip 截断 / 空文件	友好报错「不是有效的 .key 文件——Zip 结构损坏、文件被截断」
单个 .iwa 字节被改	优雅降级：× Index/Document.iwa: 不支持的 wire type 7，其余 30 个 iwa 正常解析
其他命令遇到坏 Zip	统一捕获 BadZipFile / FileNotFoundError / PermissionError，不再抛 traceback

设计原则：**文件级错误给出可读结论，段级错误只影响该文件并继续统计其余**—— 这样能区分「文件全坏」和「某个 iwa 坏了但其余可用」。

11.8 表格增删行（UID 映射的三个坑）

add-row / del-row 实测（TableSample.key 与 TableSample+row.key 对照）：

现象	结论
只改 TableModel.number_of_rows	Keynote 忽略，行数不变
只改 Tile.rowInfos	打开报缺 required 字段
追加 ColumnRowUIDMapArchive 的 UID 但不排序	**拒开**

正确做法（三处同步）：

1. sorted_row_uids 按 **upper 升序**（再按 lower）插入新 UUID；
2. row_index_for_uid：原值 ⩾ 插入位置的 +1，新条目在排序位置填插入位置；
3. row_uid_for_index：在插入位置插入新 UID 的排序下标；
4. Tile.rowInfos 同步插入，并重排所有 tile_row_index。

实测：加行后 Keynote rows=6，新行内容 = 复制源行（A3=2.0，A4 为原第 3 行）； 删行后 rows=4，后续行正确前移。

加列：每行 cell_storage_buffer 追加 24 字节空文本格 （05 03 + 8 字节 0 + 08 10 02 00 + 4 字节文本 ID + 05 00 00 00 01 00 00 00）， cell_count +1、number_of_columns +1、column 三份索引同步。实测 Keynote 5 列，新列为空。 **删列（已验证）**。Keynote 自己删一列时改了 14 个对象，逐项对齐后 del-col 通过：

1. cell_offsets 按 uint16 重算，**多余槽位填 0xFFFF**（这是关键——保留旧值会拒开）；
2. 旧格式 cell_storage_buffer_pre_bnc 每格 12 字节同步删一格，cell_offsets_pre_bnc 同样填 0xFFFF；
3. HeaderStorageBucket（列头）删一个 header、**index 连续重排**、按总和重分配宽度；
4. 行头 numberOfCells 同步改成新列数；
5. ColumnRowUIDMapArchive.column_uid_for_index：删被删列项，**原值 > pos 的都要 –1**；
6. 字符串表 TableDataList：删掉被删列引用的 key（文本格偏移 12 的 int32），**key 编号不重排**；
7. 计算引擎 FormulaOwnerDependenciesArchive.spanning_column_dependencies.total_range_for_table.bottom_right_co 收敛到新列数 – 1。

实测（删第 3 列）：Keynote 读回 rows=5 cols=3，A2=2、B2=4、C2=6 —— 正好是被删列之后的数据。

> 这一步的教训：**加列样本只能给反向线索**。0xFFFF 填充、column_uid_for_index 的 –1 重映射、 字符串表清理，都是靠「Keynote 自己删列后」的样本逐字段对比才发现的。

11.9 图片替换（data ident = 文件名后缀）

TSD.ImageArchive 的 data / thumbnailData / originalData 是 TSP.Reference， 其 ident **就是 Data/ 文件名最后一段**：

```
data: 33012  →  Data/0F6230E7-...-33012.jpg    (9.1 MB)
data: 33005  →  Data/8A076B9D-...-33005.pdf
```

set-image 直接替换该文件、同步 naturalSize / originalSize。 实测把 9.1 MB 的 JPG 换成 800×600 品红色测试图：Keynote 打开正常， 导出 PNG 品红像素 44304——**替换生效**。

11.10 超链接（url_ref 即目标）

TSWP.HyperlinkFieldArchive 只有两个关键字段：

```
super.text_attribute_uuid_string: "FFF9DB61-..."
url_ref: http://www.google.com          ← 外链
url_ref: ?slideid=43E29214-...          ← 页内跳转
```

links 列出全部链接，set-link 改 url_ref 即可。

11.11 页间转场（内联在 SlideArchive）

转场**不是独立对象**，而是内联在 SlideArchive.transition（KN.TransitionArchive）：

```
transition {
  attributes {
    animationAttributes {
      animation_type: "Transition"
      effect: "apple:magic-move-implied-motion-path"   ← none / apple:dissolve / ...
      duration: 2.0
    }
  }
}
```

transitions 列出每页转场，set-transition 改 effect 与 duration。 实测把第 2 页从 magic-move 改成 apple:dissolve + 1.5s，读回一致、文件 verify 通过、Keynote 正常打开。

11.12 新增构建动画

KN.BuildArchive 只有五个字段：

```
drawable: { identifier: 7030297 }      ← 动画作用的对象
delivery: "All at Once"
duration: 0.0
attributes {
  eventTrigger: 1
  animationAttributes {
    animation_type: "In"
    effect: "apple:bc-appear"          ← 效果名
    duration: 1  delay: 0  random_number_seed: ...
  }
}
chunk_id_seed: 1
```

KN.BuildChunkArchive 引用 build，并带自己的 build_chunk_identifier（UUID 对）。

新增做法：复制一个现有 BuildArchive，把 drawable 换成目标对象、effect 换成目标效果； 再复制一个 BuildChunkArchive，把 build 指向新 build、重新生成 UUID； 最后把两个引用追加到 SlideArchive.builds / SlideArchive.buildChunks。

实测：给 AnimSample.key 第 2 页的对象 0 加 apple:bc-appear， builds-list 显示 3 个动画、文件 verify 通过、Keynote 正常打开（slides=2）。

> 局限：文件层与「Keynote 能打开」已确认，但**放映时动画是否真的播放**尚未在 Keynote UI 里逐一点验，交付前建议手动放映确认。

11.13 样式库编辑（批量生效）

样式对象自带 super.name 与 super.style_identifier，改一处会影响所有引用它的文本。

styles 列出全部样式（ident / 类型 / 名称 / style_identifier / 关键属性）：

```
1611294  字符  红色（粗体）        bold=True color=#C82505
1611176  字符  链接              character-style-hyperlink
1611213  段落  –               chart-0-paragraphStyle_0   size=13 color=#000000
```

set-style 改单个属性（font_size / font_color / bold / italic）：

```
$                          set-style              out.key 6882870 font_color=#FF00FF
样式 6882870「金句」：font_color None → #FF00FF
→ 导出 PNG：品红 3653（与 set-text-color 整段改色一致）
```

> 与 set-text-color 的分工：set-style 是**批量**（所有引用该样式的页一起变）， set-text-color 是**页级精确**（新建 CharacterStyle，不污染共享样式）。

11.14 性能实测

文件	大小	解析	对象数	每对象	写入（含进程启动）
模板 1 页	0.4 MB	22 ms	1360	16 µs	168 ms
母版模板 22 页	1.0 MB	31 ms	2635	12 µs	183 ms
设计规范 42 页	1.5 MB	57 ms	5681	10 µs	252 ms
T.key 1 页图表	0.5 MB	23 ms	1361	17 µs	—

解析与对象数线性相关；按 42 页 57 ms 外推，**1000 页约 1.4 秒解析、3–5 秒写入**， 对工具链不是瓶颈。

11.15 一处诊断经验：别把「打开失败」当成文件坏了

调试期间多次出现 Keynote「打不开」，实际是 **AppleScript open POSIX file 静默失败** （–1719 / –1712），改用 open –a "Keynote Creator Studio" <file> 即可正常打开。 判断文件是否真的损坏，应以 verify + open –a + 读回字段三者交叉为准。

十二、实机验证：二十三种改法全部通过

场景	改动	结果
等长替换	版式名 _纯黑背景 → _纯白背景（34→34 字节）	Keynote 打开正常，读到新名
变长替换	版式名 _纯黑背景 → _纯黑色背景（34→**37** 字节）	Keynote 打开正常，读到新名
跨文件批量	**17 个 .iwa 的页脚 → 2026（每处 +5 字节）**	**17 个文件、17 处全部生效；Keynote 打开正常并读到改后页脚**
追加段落	**第 1 页备注末尾追加一段（185→218 字，段落数 +1）**	**Keynote 打开正常，presenter notes 读出 218 字且含追加内容**
跳过某页	**第 3 页 isSkipped=1**	**Keynote 读出「被跳过的页: 3,12」——新写入的 3 与原生的 12 同时生效**

场景	改动	结果
取消跳过	第 12 页恢复 isSkipped=0（显式写 0）	Keynote 读出「被跳过的页: (无)」
批量跳过	2–4,6 四页一次写入	Keynote 读出「被跳过的页: 2,3,4,6,12」
整段改写正文	第 1 页标题 8→15 字（非子串替换）	Keynote 文本对象中读到改写后的标题
覆盖备注	第 1 页备注 185→36 字	Keynote presenter notes 读出新内容
替换媒体	替换 Data/mt-*.jpg 字节（文件名不变）	Keynote 正常打开（slides=1, layouts=23）
表格文字	"指标" → "对比指标"（+6 字节，在 Index/Tables/DataList.iwa）	Keynote 第 17 页读出「对比指标 / recall@8 / 0.929」
动画时长与自动播放	某块 时长 0.5→3.0、自动 False→True	回读一致，Keynote 打开正常（slides=31）
综合体检	对规范 deck 与历史 deck 各跑一次	规范 deck 全绿；历史 deck 报出 23 页缺备注、8 页带动画
表格数值	第 2 行第 2 列 0.929 → 0.777（整数 929 → 777）	Keynote 读出 0.777
文字颜色	单页样本 font_color 绿→紫	导出 PNG 中绿色消失、紫色出现（AppleScript 读回是缓存旧值，不可信）
页背景色	第 2 页 SlideStyleArchive.fill.color 白→红	PNG 第 2 页变 (252,59,47)，第 1 页仍为白——只影响目标页
形状填充色	母版模板共享样式 734 红→绿	9 个引用页的红色块同时变绿，其余颜色不动
删除幻灯片	母版模板第 5 页	22 → 21 页；Keynote 正常打开
复制幻灯片	第 1 页复制到其后	22 页；新页与原页共享绘制对象
按版式新增页	add-slide "标题项目符号"	1 → 2 页；深拷贝 24 对象；改新页不影响旧页
删除形状	第 1 页第 3 个形状（级联删 storage/caption）	形状 3 → 2；Keynote 正常
复制形状	第 1 页第 1 个形状	形状 3 → 4；Keynote 正常
页级精确改文字色	新建 CharacterStyle 挂 table_char_style	两页样本：目标页 1655 品红，对照页 0；共享样式未变
文本框改色	drawable 路径 2（含群组 3/1）	3211 像素变化、品红 1730
插入形状	dup-shape --offset 0,250	文件可打开、渲染 1459→1648、副本独立
打组 / 拆组	group / ungroup	渲染 1944、与原图差异 0

场景	改动	结果
audit --fix	audit --fix --out	1 页缺备注 → 0 页
局部改色	--substr / --range	单字级生效，像素量与字数匹配
备注子串替换	set-note --replace	只替换目标子串，其余备注保留
改写既有备注	空备注先写、再整段替换	Keynote 读回第二次内容

一处值得记录的踩坑

变长替换的第一版实现**只更新了 MessageInfo.length 和 ArchiveInfo 前缀**，结果：

- Keynote **能打开文档**（layouts=23），
- 但那个版式名**读不出来**（返回 missing value）。

原因：**object 内部**包裹字符串的 protobuf 字段长度没改。补上"递归下钻、逐层重算"后才正确。教训是：**"能打开"不等于"改对了"**，必须回读被改字段本身。

十三、可操作性边界

操作	状态	说明
解包 / 读结构	已验证	Zip + IWA + Snappy
语义识别对象	已验证	1360/1360 解析，631 类型映射
提取真实文字	已验证	按对象定位，含页脚/正文/版式名
按页导出大纲	已验证	标题 / 正文 / 备注（21 页 deck）
等长替换	已验证	不触碰任何长度字段
变长替换	已验证	长度链自动重算
跨文件批量替换	已验证	实测 17 文件 / 17 处
追加演讲者备注	**已验证**	**按页追加段落，Keynote 实机确认可读**
跳过 / 取消跳过某页	**已验证**	**isSkipped(4)；三向验证 + 批量范围语法**
整段改写标题 / 正文	**已验证**	沿引用链改 StorageArchive.text，非子串替换
改写既有备注	**已验证**	set-note 整段替换；--replace 只替换子串
备注合规体检	**已验证**	实测 21 页 deck，全部合规
替换媒体文件	**已验证**	同名字节替换，无元数据需同步
表格文字写入	**已验证**	改 Index/Tables/DataList.iwa 的 stringTable
表格数值写入	**已验证**	cell_storage_buffer +12 的缩放整数，Keynote 读回 0.777
图表数值	同表格机制	数据源同构；未单独实测

操作	状态	说明
构建动画读写	**已验证**	builds 读；set-build 改时长/自动播放
颜色实例枚举	**已验证**	colors 递归列出全部内联 TSP.Color（ 模板 2939 个）
改颜色（按序号）	**已验证**	set-color；Keynote 打开 + PNG 像素双验证
改某页背景	**已验证**	set-bg；第 2 页白→红，第 1 页不受影响
改某页形状填充	**已验证**	set-shape；共享样式时提示影响的页数
文字色的页级精确改	**已验证**	新建 CharacterStyle 挂到 table_char_style（已落地）；支持整段 / --substr / --range / --para-index，目标含标题、正文与任意文本框
综合体检	**已验证**	audit 汇总备注/跳过/动画/表格
删除 / 复制幻灯片	**已验证**	del-slide / dup-slide；页序由 slideTree.slides 决定
按版式新增页	**已验证**	add-slide；深拷贝版式闭包，改内容互不影响
删除 / 复制形状	**已验证**	del-shape / dup-shape；删除会级联清理 storage/caption
页级精确改文字色	**已验证**	set-text-color 支持 title/body 与文本框路径；默认字符级覆盖
改写既有备注	**已验证**	set-note；整段替换或 --replace 子串替换

推荐路径：

- **改文字**（标题、正文、页脚、版式名、备注）→ 用 ，快且可脚本化、可批量。
- **结构性修改**（增删页、换版式、插入形状）→ 用 **AppleScript** 驱动 Keynote 本体。

十四、风险与限制

1. **schema 来自版本对齐**：本次用 v14_5 定义（对应 Keynote 14.5）。跨大版本升级后类型编号可能变化，届时需更新 schema。
2. **跨版本可能失效**：改完务必先用副本、再用 Keynote 打开确认。
3. **始终保留原件**。
4. **只能改已存在的字符串内容**：不能通过替换凭空插入新对象（引用图会不一致）。
5. **写回用真 Snappy 压缩**：iwa_wrap 会做贪心 LZ77 压缩并做 round-trip 自校验， 失败自动退回 literal。实测改后 DocumentStylesheet 为 47,004 字节， 与原生 47,858 基本一致（旧版 literal 写法会膨胀到 305,744 字节）。
6. **改样式 ≠ 改某一页**：颜色挂在共享样式上；改一个样式会同时影响所有引用它的页， 工具会打印受影响的页列表。

十五、附录：工具与用法

配套脚本 （读写核心仅依赖 Python 3 标准库；语义命令额外需要 protobuf 与固化的 kn_schema）。

```
# 容器与校验
                    list   <file.key>
                    verify <file.key>

# 无 schema 的文字搜索（纯字节层）
                    text   <file.key> [关键字]

# 语义层
                    types   <file.key>           # 按类型统计对象
                    texts   <file.key> [关键字]  # 列出真实文字及所在对象
                    outline <file.key>           # 按页导出：标题/正文/备注

# 备注合规体检（默认要求含【内容依据】【设计依据】）
                    check-notes <file.key> [必需字段...]

# 在备注末尾追加一段（保留既有内容）
                    note     <src.key> <dst.key> <页码> <要追加的文字>

# 改写某页备注（整段替换；--replace |旧文本 只替换子串）
                    set-note <src.key> <dst.key> <页码> <新备注全文>

# 整段重设某页标题 / 正文
                    set-text <src.key> <dst.key> <页码> <title|body> <新文字>

# 跳过 / 取消跳过（页码表达式支持 3 / 1-3,7 / all）
                    skip     <src.key> <dst.key> <页码表达式> [--off]

# 对象级内容
                    audit    <file.key> [--fix]    # 综合体检
                    builds   <file.key>            # 列出各页构建动画
                    set-build <src> <dst> <块ident> [duration=2.5] [automatic=true]

# 颜色（TSP.Color 是内联子消息）
                    colors   <file.key> [关键字] [--all]  # 列出颜色实例
                    set-color <src> <dst> <序号> <#RRGGBB|r,g,b> [--alpha a]
                    set-bg    <src> <dst> <页码> <#RRGGBB>       # 页背景
                    set-shape <src> <dst> <页码> <形状路径> <#RRGGBB>
                    set-text-color <src> <dst> <页码> <title|body|形状路径> <#RRGGBB> \
                    [--substr 文字|--range a:b] [--para]
                    set-note <src> <dst> <页码> <新文本> [--replace |旧文本]

# 结构编辑
```

```
                    add-slide <src> <dst> <版式名> [位置]
                    del-slide <src> <dst> <页码表达式>
                    dup-slide <src> <dst> <页码>
                    del-shape <src> <dst> <页码> <形状路径>
                    dup-shape <src> <dst> <页码> <形状路径>

# 替换（默认等长；--resize 变长；--all 跨全部 .iwa）
                    edit   <src.key> <dst.key> <iwa名|-> <旧串> <新串> \
                         [--resize] [--all]
```

实测输出：

```
$                    types
对象总数 1360（已识别 1360 / 未知类型 0）

$                    outline                                .key
共 21 页

--- 第 1 页 ---
  标题:
  正文: 一次真实自建的选择与权衡
  备注: 有（185 字）
...
统计：21 页，其中 21 页有演讲者备注

$                    texts          版权所有
含文字的文本对象 17 个（关键字 '版权所有'）
  [  6879323] TemplateSlide.iwa  1段  '                    '

$                    edit           out.key - \
    "                    " "                    2026" --resize --all
已写出 out.key（509136 字节；17 个 .iwa 被修改，共替换 17 处）
```

操作纪律：先 texts 定位 → 对**副本**执行 edit（脚本回读校验）→ **Keynote 打开确认** → 再替换原件。

schema 固化位置

```
~/Library/Python/3.15/lib/python/site-packages/kn_schema/  (v14_5, 约 1.6 MB)
```

由 keynote-parser 1.14.5 的 v14_5 定义**改写命名空间**而来（不占用 keynote_parser，与原包无冲突），只含类型映射与 pb2，**不引入 snappy/yaml 等重依赖**。

十六、方法学说明

三条证据线互相印证：

1. **构件分析**：（61 节点 / 60 边，integrity_contradictions: 0）。
2. **自建解码器**：Snappy、varint、protobuf 节点化/重编码、段模型切分全部自行实现。 段模型先经错误假设证伪，再由全量解析（31 文件 / 1354 段 / 1360 对象）确认。
3. **官方 schema**：keynote-parser 的 codec.py 序列化表达式 [_VarintBytes(header.ByteSize()), header.SerializeToString()] + [obj.SerializeToString() ...] 独立印证段结构；其 v14_5 pb2 定义提供对象类型映射，并实测 1360/1360 解析成功。

验收判据始终是 Keynote 14.5 实机打开：等长、变长、跨文件批量三种改法均已通过。